Chapter 12 – Case Studies (IV): Game Problems

IERG4320/IEMS5726 Data Science in Practice | Cheat Sheet

Game AI Overview

游戏AI：用搜索和学习找到最优策略

Games & AI: well-defined rules (env), actions, goals (rewards). AI serves as opponents, teammates, commentators, story guides.

Search techniques: A* search, alpha-beta pruning (minimax), fuzzy logic, genetic algorithm.

搜索技术：A*、Alpha-Beta剪枝、模糊逻辑、遗传算法

Rule-based: if winning move → win; if opponent winning → block; else promote or random. Simple but limited for complex situations.

Blackjack Environment

21点：不超过21点的前提下赢过庄家

Objective: beat dealer without going over 21.
Cards: 2–10 face value; J/Q/K = 10; Ace = 1 or 11.

State: tuple (player_sum, dealer_showing, usable_ace).
E.g. (15, 10, False): player has 15, dealer shows 10, no usable Ace.

Actions: Hit (take card) or Stand (stop).

Reward: +1 win, –1 loss, 0 draw (sparse, end of episode only).

Dealer policy: must hit until sum ≥ 17.

庄家必须在<17时继续要牌

Basic strategy: win 42.2%, loss 49.1%, tie 8.5%.

D3QN (Dueling Double DQN)

D3QN：双重DQN减少过估计 + 对决架构分离状态价值和优势

Double DQN

双重DQN：用在线网络选动作，目标网络评估

Fixes overestimation bias in standard DQN.
Standard: $y = r + \gamma \max_{a'} Q_{tar}(s', a'; \theta^-)$
Double: $y = r + \gamma\, Q_{tar}(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$

用θ选动作，用θ^-评估价值

Dueling Architecture

对决架构：分成价值流V(s)和优势流A(s,a)

Splits Q-network into two streams:
Value $V(s)$: how good is this state?
Advantage $A(s,a)$: how much better is action a than average?
$Q(s,a) = V(s) + \big(A(s,a) - \text{mean}_a A(s,a)\big)$
E.g. state (20,2,False): $V(s)$ high (great position), $A(\text{STAND})$ positive, $A(\text{HIT})$ very negative.

Loss: $\mathbb{E}[(y - Q(s,a;\theta))^2]$ where θ^- updated periodically.
Result: win rate ≈ 43–44% after 10k games.

A3C (Async Advantage Actor-Critic)

A3C：多个并行worker异步更新全局网络

vs DQN: supports continuous actions, trains faster, avoids maximization bias.

Key Components

- **Asynchronous**: multiple workers explore in parallel, decorrelate data naturally 并行探索，自然去相关
- **Actor**: learns policy $\pi(a|s)$ 策略网络
- **Critic**: learns value $V(s)$ 价值网络
- **Advantage**: $A(s,a) = Q(s,a) - V(s)$
  Positive → reinforce; Negative → discourage 正=强化，负=抑制

Training Pipeline

1. Workers collect (s, a, r) sequences independently
2. Compute n-step returns → estimate advantage
3. Calculate actor + critic gradients
4. Asynchronously update global network

n-step回报：短n=高偏差低方差；长n=低偏差高方差

Result: win rate ≈ 38–40% (tends to always STAND).

AlphaZero / AlphaBlackjack

AlphaZero：DeepMind开发，只用规则+自我对弈学习

Self-play only: given rules, plays millions of games against itself.

MCTS (Monte Carlo Tree Search)

蒙特卡洛树搜索：选择→扩展→模拟→反向传播

1. **Selection**: pick node via $Q + U$ (exploit + explore)
2. **Expansion**: add new move to tree
3. **Simulation**: play randomly to end (rollout)
4. **Backpropagation**: update stats along path

Dual-Headed Network

Policy head (Actor): outputs $[P(\text{STAND}), P(\text{HIT})]$.
Value head (Critic): scalar $\in [-1, +1]$.
Training: policy gradient (actor) + MSE (critic).

Self-Play Training

Early: learn basics (don't hit on 20).
Middle: nuanced strategy (16 vs dealer's 7).
Late: master complex situations (soft hands).
Each iteration plays against previous version.
Result: win rate ≈ 38–40%.

Single Agent for Multiple Games

单智能体玩多个游戏：核心挑战是灾难性遗忘

Problem: catastrophic forgetting – learning Game B overwrites weights crucial for Game A.

灾难性遗忘：学新游戏会覆盖旧游戏的重要权重

EWC (Elastic Weight Consolidation)

弹性权重固化：惩罚对旧任务重要参数的大改动

Penalizes large changes to parameters important for previous tasks; allows flexibility for unimportant ones.

Process: train Game A → train Game B with EWC (important weights "elasticated" toward A's solution) → repeat for Game C (EWC terms for both A and B).

Target games: Blackjack (simple), Leduc Hold'em (imperfect info, bluffing), Hearts (long-term strategy).

3D Model Generation

3D模型生成：从2D图像或文本生成3D模型

3D representations: mesh (vertices + faces), depth map, voxel grid, point cloud, implicit surface.

Triangle mesh: standard for graphics, adaptive, renderable, less memory. 8 vertices + 12 faces = cube.

```python
import trimesh, numpy as np
vertices = np.array([
  [0,0,0],[1,0,0],[1,1,0],[0,1,0], # bottom
  [0,0,1],[1,0,1],[1,1,1],[0,1,1]  # top
])
faces = np.array([
  [0,1,2],[0,2,3], [4,5,6],[4,6,7], # bot/top
  [0,4,7],[0,7,3], [1,5,6],[1,6,2], # L/R
  [0,1,5],[0,5,4], [3,2,6],[3,6,7]  # F/B
])
mesh = trimesh.Trimesh(vertices, faces)
mesh.show()
```

TripoSR (2D → 3D)

TripoSR：单张图片重建3D模型

Input: single RGB image. Output: 3D mesh.
Pipeline: DINOv1 encoder → Transformer decoder → triplane NeRF → marching cubes → mesh.
No camera params needed (learned during training).

Improvements: 40-channel triplane, BCE mask loss (reduce floaters), local 128×128 patch rendering.
Metrics: Chamfer Distance (lower=better), F-Score (higher=better).

Shap-E (Text → 3D)

Shap-E：文本/图像生成3D隐式神经表示，约13秒

Generates INR (Implicit Neural Representation) parameters via diffusion.
Two-stage: (1) text → 2D image (image diffusion); (2) image → 3D point cloud (point cloud diffusion conditioned on CLIP embedding).
Upsampler: low-res → high-res point cloud → Poisson surface reconstruction → mesh.
Speed: ∼13s per asset on V100 GPU.
Limitations: lower fidelity, floater artifacts, topological errors, bottlenecked by 2D model quality.

Robot Learning

机器人学习：构建感知-动作循环

Robot vision: embodied, active, situated.
Formulated as RL: perception → action → feedback loop.

Transfer Learning

迁移学习：用预训练模型适应新任务

- **Fixed feature extractor**: freeze pre-trained layers, train new classifier
- **Fine-tuning**: continue backprop on pre-trained weights
- **When**: small+similar → fix features; large+similar → fine-tune all; small+different → earlier layers; large+different → fine-tune all

Robot Learning Methods

机器人学习方法

- **Imitation Learning**: mimic expert demonstrations 模仿学习
- **Behavior Cloning**: supervised (state → action) 行为克隆
- **Inverse RL**: infer expert's reward function, then optimize 逆强化学习
- **Diffusion Policies**: generate multi-modal action sequences via diffusion models 扩散策略

Ch12 Summary – 3 Perspectives:
Learn to play: D3QN, A3C, AlphaZero, EWC
Learn to design: TripoSR (2D→3D), Shap-E (text→3D)
Learn to walk: Transfer learning, imitation, behavior cloning, inverse RL, diffusion policies